

编译原理

3. 语法分析-CFG及其解析

rainoftime.github.io

浙江大学

计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
- 3. Parsing**
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

本讲内容

- 1 语法分析概述
- 2 上下文无关文法
- 3 设计编程语言的文法

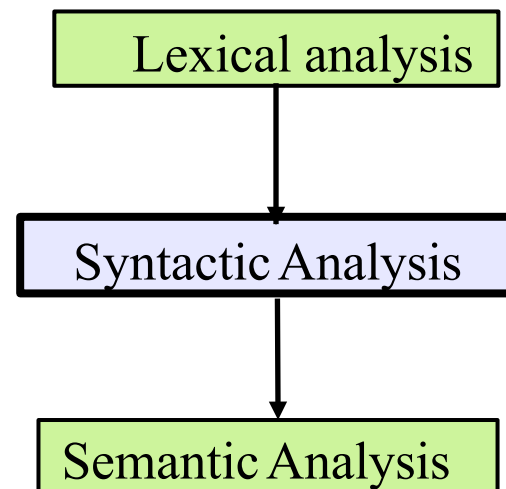
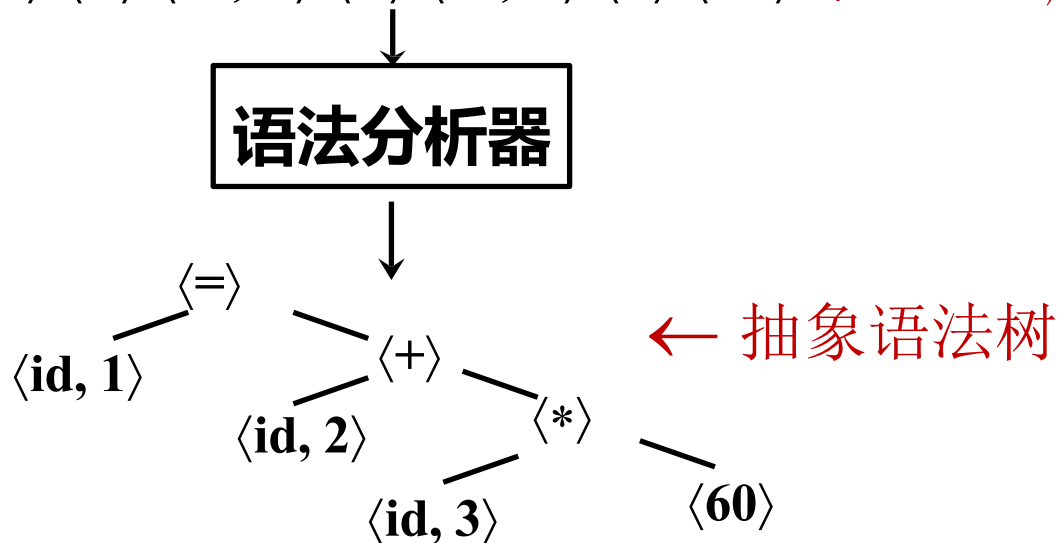
1. 语法分析概述

什么是语法分析？

回顾: 语法分析器的作用

- 从词法分析器获得Token序列，确认其是否可以由**语言的文法**生成
 - 对于语法错误的程序，报告错误信息; 对于语法正确的程序，生成**语法分析树**
 - 通常产生的是**抽象语法树**（AST）

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle \leftarrow \text{Token序列}$



例: 简单计算器程序

$1 + (2 - 3)$ 合法

$1 + 2 - 3 +$ 非法

$1\ 2 + - 3$ 非法

$1 + 2 - a$ 非法



例: C语言语法错误

- 以下程序没有词法错误，但是有多个语法错误

```
int foo(int x){  
    int y;  
    if ((x > 0)  
        y = 1;  
    else  
        y = 0  
    return y;  
}
```

foo.c:4:9: **error:** expected ')'
 y = 1;
 ^

foo.c:3:8: **note:** to match this '('
 if ((x > 0)
 ^

foo.c:6:14: **error:** expected ';' after expression
 y = 0
 ^
 ;
 ^

2 errors generated._

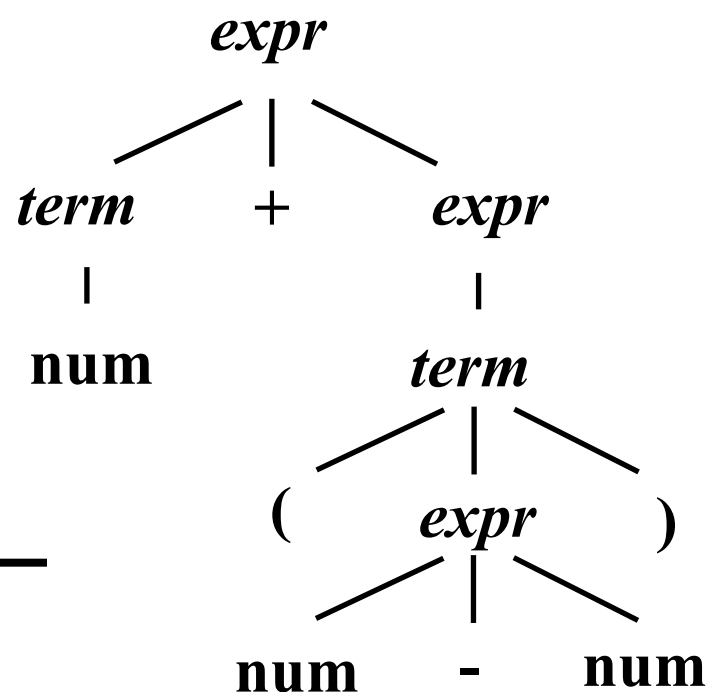
如何判定输入合法性 & 生成语法树

- 首先规定合法的**基本单元**—词法分析
 - Token: 如由0-9组成的数字(num)和符号+、-、(、)

- 其次要理解算术表达式的构成

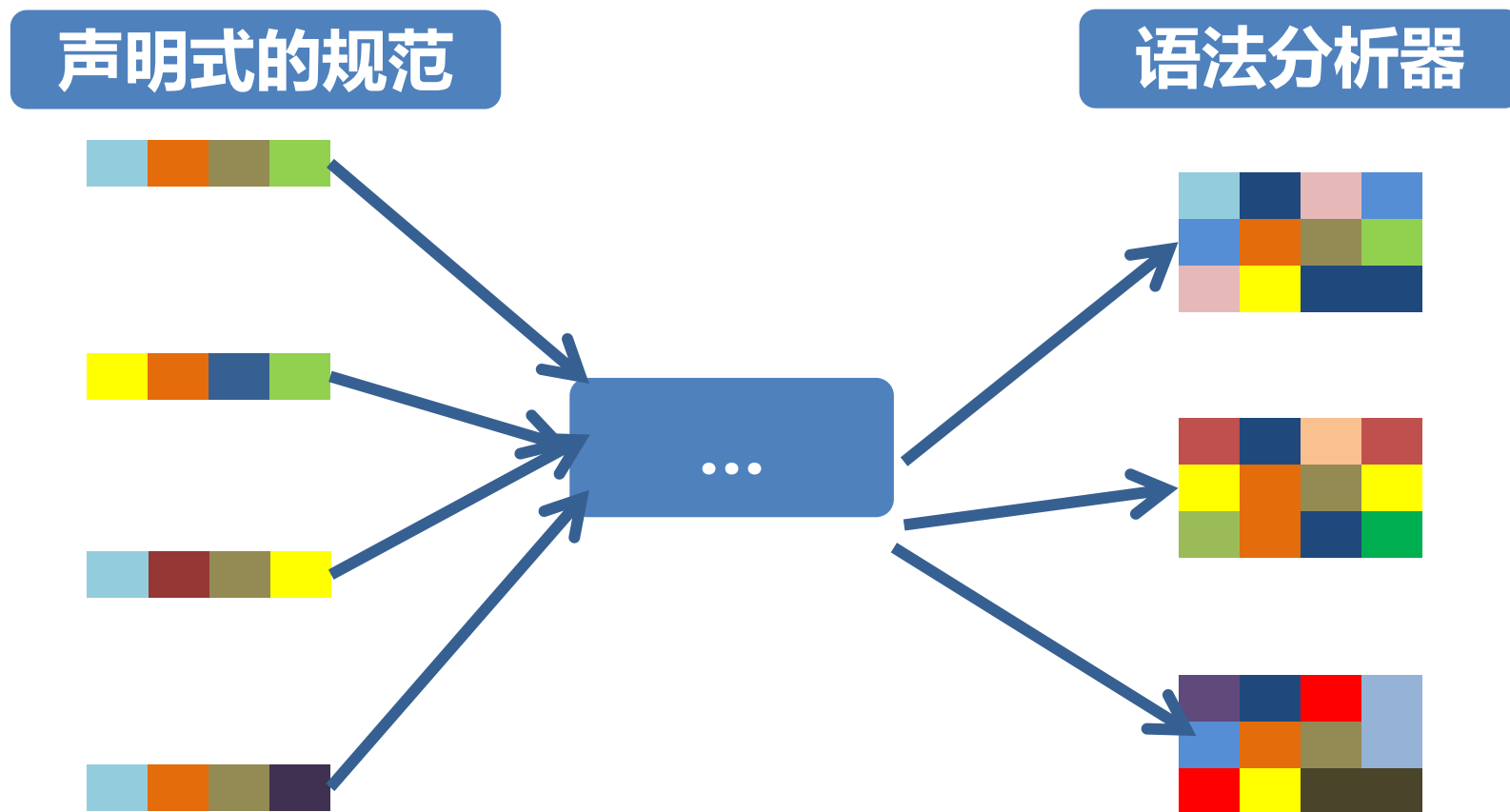
- 大表达式可拆为子表达式
- 直至看到的都是**基本单元**

Parse Tree



语法分析器的构造

Program = **Specification** + **Implementation**
“What” “How”



Specifying the syntax with **Context-Free Grammars (CFG)**!

语法分析器的实现方式

- **Writing Manually (recursive descent)**
 - Clang, gcc (Since 3.4)
 - Libraries can make this easier (e.g., parser combinators)
- **Using a Parser Generator**
 - Gcc (Before 3.4), Glasgow Haskell Compiler, OCaml
 - Parser generator: Yacc, Bison, ANTLR, menhir

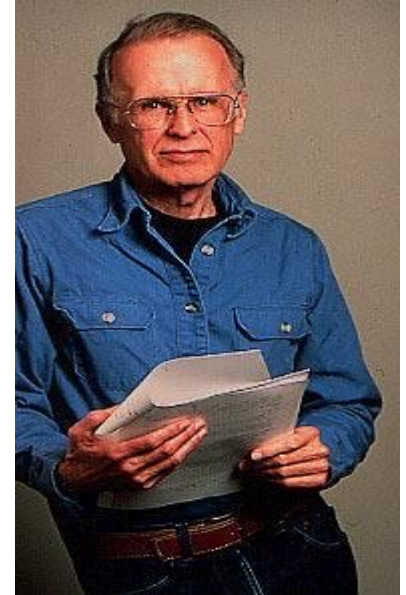
2. 上下文无关文法

- CFG简介
- 推导和归约
- CFG的Parse Tree

如何形式化定义编程语言的语法？

上下文无关文法 (Context-Free Grammar)

- **John Backus(1977图灵奖)提出**
 - **提出了多种高级编程语言**
 - Speedcoding -> FORTRAN ->ALGOL 58 -> ALGOL 60
 - **提出了编译技术的理论基础**
 - 巴科斯范式 (Backus–Naur Form)
 - **上下文无关文法**
 - **对计算机科学影响巨大**
 - 诞生了许多理论研究成果
 - 现代编译器还保留了FORTRAN I的大概架构



弗吉尼亚大学化学专业，哥伦比亚大学数学专业，曾服务于阿波罗登月计划

上下文无关文法: 描述语言的语法结构

$$G = (T, N, P, S)$$

T : 终结符集合 (Terminals)

N : 非终结符集合 (Non-terminals)

P : 产生式集合 (Productions) $A \rightarrow \alpha, A \in N, \alpha \in (T \cup N)^*$

S : 开始符号 (Start symbol): $S \in N$

上下文无关文法: 终结符

$$G = (T, N, P, S)$$

T : 终结符集合 (Terminals): 组成串的基本符号(*token*)

– 例: $T = \{ \text{num}, +, -, (,) \}$

- Once generated, terminals are permanent
- Terminals ought to be tokens of the language

上下文无关文法: 非终结符

$$G = (T, N, P, S)$$

T : 终结符集合 (Terminals)

N : 非终结符集合 (Nonterminal): 表示串的集合的语法变量

- 在程序语言中通常对应于某个程序构造
 - 例: $N = \{\text{expr}, \text{term}, \text{stmt}\}$

$$T \cap N = \Phi$$

$T \cup N$: 文法符号集

上下文无关文法: 产生式

$$G = (T, N, P, S)$$

T : 终结符集合 (Terminals)

N : 非终结符集合 (Nonterminals)

P : **产生式** $A \rightarrow \alpha$ **集合** (Productions): $A \in N, \alpha \in (T \cup N)^*$

- 头(左)部 A 是一个**非终结符号**, 右部 α 是一个符号串, 描述将终结符和非终结符组合成串的方法

$$E \rightarrow E + E,$$

$$E \rightarrow E * E,$$

$$E \rightarrow (E),$$

$$E \rightarrow \text{id}$$

注：产生式(Productions)的缩写

- 对一组有**相同左部**的 α 产生式

$$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$$

- 可以简记为

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- 读作： α **定义为** β_1 ，或者 β_2 ，...，或者 β_n

$\beta_1, \beta_2, \dots, \beta_n$ 称为 α 的**候选式**(Candidate)

➤ 例

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{id}$$



$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

上下文无关文法: 开始符号

$$G = (T, N, P, S)$$

T : 终结符集合 (Terminals)

N : 非终结符集合 (Non-terminals)

P : 产生式集合 (Productions) $A \rightarrow \alpha, A \in N, \alpha \in (T \cup N)^*$

S : **开始符号** (Start symbol): $S \in N$ 唯一的一个开始符号

- 某个被指定的**非终结符号**

- 它对应的串的集合, 就是文法的语言

例: 上下文无关文法

$expr \rightarrow expr\ op\ expr$

$expr \rightarrow (expr)$

$expr \rightarrow -\ expr$

$expr \rightarrow \mathbf{id}$

$op \rightarrow + \mid - \mid * \mid /$

- 非终结符集合 $N = \{expr, op\}$
- 终结符集合 $T = \{+, -, *, /, (,), id\}$
- $expr$ 是开始符号

例: 上下文无关文法

- | | | |
|---------------------------------------|-------------------------------|-------------------------|
| 1. $S \rightarrow S; S$ | 4. $E \rightarrow \text{id}$ | 8. $L \rightarrow E$ |
| 2. $S \rightarrow \text{id} := E$ | 5. $E \rightarrow \text{num}$ | 9. $L \rightarrow L, E$ |
| 3. $S \rightarrow \text{print} (L)$ | 6. $E \rightarrow E + E$ | |
| | 7. $E \rightarrow (S, E)$ | |

Grammar 3.1. A syntax for straight-line programs. (From Tiger Book)

- $T = \{\text{id}, \text{print}, \text{num}, +, (,), \}, N = \{S, E, L\}$
- Start symbol: S

Strings/sentences in the language of this CFG:

- $\text{id} := \text{num}$
- $\text{id} := \text{num} + \text{num}$
- $\text{print} (\text{num})$
- $\text{id} := \text{id} + (\text{id} := \text{num} + \text{num}, \text{id})$
-

上下文无关文法 – EOF Marker

- $\$$: end of file (EOF)
- To indicate that $\$$ must come after a complete S -phrase
 - Add a new start symbol S' and a new production $S' \rightarrow S\$$

$S \rightarrow E\$$

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow \text{id}$

$F \rightarrow \text{num}$

$F \rightarrow (E)$

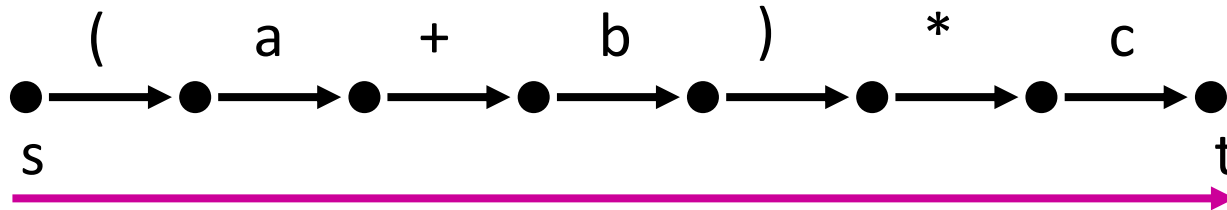
Grammar 3.10. (Tiger book)

引申: CFG只能表达 “语法” 吗? (不要求掌握)

- **CFL-Reachability** (结合CFG和图可达性)
 - G : Graph (N nodes, E edges); L : A context-free language
 - L -path from s to t iff $s \xrightarrow{\alpha}^* t, \alpha \in L$
 - Running time: $O(N^3) \rightarrow O(N^3/\log N)$ [POPL'08]

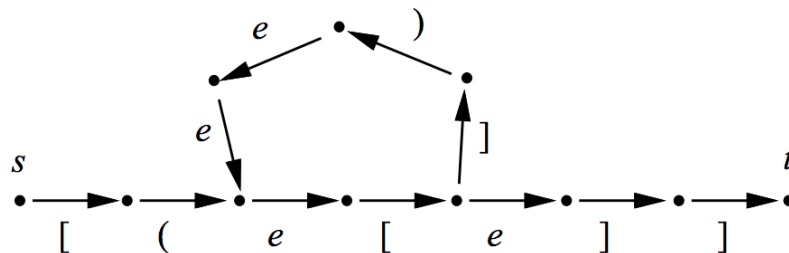
$\text{exp} \rightarrow \text{id} \mid \text{exp} + \text{exp} \mid \text{exp} * \text{exp} \mid (\text{exp})$

“(a + b) * c” $\in L(\text{exp})$?



引申: CFG只能表达“语法”吗? (不要求掌握)

- **CFL-Reachability** (结合CFG和图可达性)
 - G : Graph (N nodes, E edges); L : A context-free language
 - L -path from s to t iff $s \xrightarrow{\alpha}^* t, \alpha \in L$
 - Running time: $O(N^3) \rightarrow O(N^3/\log N)$ [POPL'08]
- **CFL-Reachability用于分析程序语义**
 - 类型推导、别名分析、数据流分析、etc.
 - E.g., LLVM内部的别名分析cfl-aa



$L: matched \rightarrow matched\ matched$
| $(\ matched\)$
| $[\ matched\]$
| e
| ϵ

Only one L-Path : $[(e[])eee[e]]$

1. 上下文无关文法

- CFG简介
- **推导和归约**
- CFG的Parse Tree

给定文法，如何判定输入串属于文法规定的语言？

推导 (Derivations) 和归约(Reductions)

给定文法 $G = (T, N, P, S)$

- **直接推导**: 把产生式看成**重写规则**, 把符号串中的非终结符用其产生式右部的串来代替

如果 $A \rightarrow \gamma \in P$, 且 $\alpha, \beta \in (T \cup N)^*$, 称串 $\alpha A \beta$ **直接推导出** $\alpha \gamma \beta$, 并记作 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。

- **直接规约**: 如果 $\alpha A \beta \Rightarrow \alpha \gamma \beta$, 则 $\alpha \gamma \beta$ **直接规约到** $\alpha A \beta$

推导 (Derivations) 和归约 (Reductions)

给定文法 $G = (T, N, P, S)$

- **多步推导**: 如果 $\alpha_0 \Rightarrow \alpha_1, \alpha_1 \Rightarrow \alpha_2, \alpha_2 \Rightarrow \alpha_3, \dots, \alpha_{n-1} \Rightarrow \alpha_n$, 则可以记作 $\alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \alpha_3 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n$,

称符号串 α_0 **经过 n 步推导** 出 α_n , 可简记为 $\alpha_0 \Rightarrow^n \alpha_n$

– $\Rightarrow^+$ 表示“经过正数步推导”

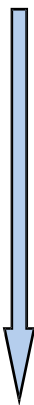
– $\Rightarrow^*$ 表示“经过若干 (可以是0) 步推导”

最左推导 (Left-most Derivation)

- **最左推导**: 每步代换**最左边的非终结符**

文法
① $E \rightarrow E + E$
② $E \rightarrow E * E$
③ $E \rightarrow (E)$
④ $E \rightarrow \text{id}$
输入
id + (id + id)

最左
推
导



推导过程

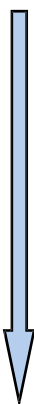
$$E \Rightarrow_{lm} E + E$$

最左推导 (Left-most Derivation)

- **最左推导**: 每步代换**最左边的非终结符**

文法
① $E \rightarrow E + E$
② $E \rightarrow E * E$
③ $E \rightarrow (E)$
④ $E \rightarrow \text{id}$
输入
id + (id + id)

最左推导



推导过程

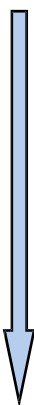
$$\begin{aligned} E &\Rightarrow_{lm} E + E \\ &\Rightarrow_{lm} \text{id} + E \end{aligned}$$

最左推导 (Left-most Derivation)

- **最左推导**: 每步代换**最左边的非终结符**

文法
① $E \rightarrow E + E$
② $E \rightarrow E * E$
③ $E \rightarrow (E)$
④ $E \rightarrow \text{id}$
输入
id + (id + id)

最左推导



推导过程

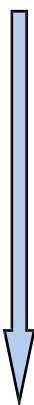
$$\begin{aligned} E &\Rightarrow_{lm} E + E \\ &\Rightarrow_{lm} \text{id} + E \\ &\Rightarrow_{lm} \text{id} + (E) \end{aligned}$$

最左推导 (Left-most Derivation)

- **最左推导**: 每步代换**最左边的非终结符**

文法
① $E \rightarrow E + E$
② $E \rightarrow E * E$
③ $E \rightarrow (E)$
④ $E \rightarrow \text{id}$
输入
$\text{id} + (\text{id} + \text{id})$

最左推导



推导过程

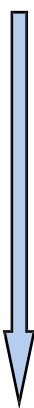
$$\begin{aligned} E &\Rightarrow_{lm} E + E \\ &\Rightarrow_{lm} \text{id} + E \\ &\Rightarrow_{lm} \text{id} + (E) \\ &\Rightarrow_{lm} \text{id} + (E + E) \end{aligned}$$

最左推导 (Left-most Derivation)

- **最左推导**: 每步代换**最左边的非终结符**

文法
① $E \rightarrow E + E$
② $E \rightarrow E * E$
③ $E \rightarrow (E)$
④ $E \rightarrow \text{id}$
输入
id + (id + id)

最左推导

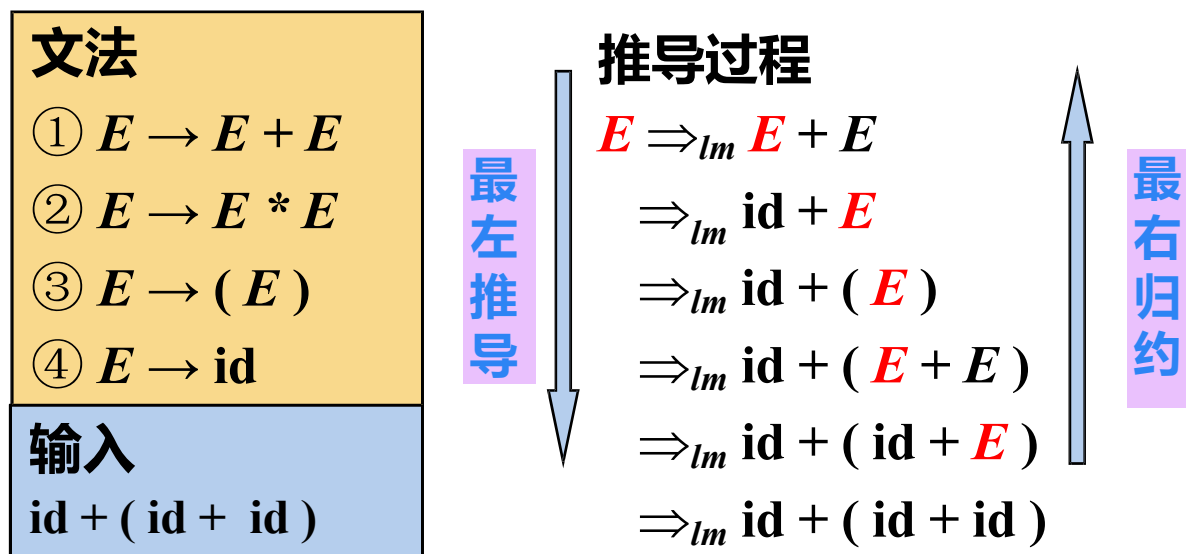


推导过程

$$\begin{aligned} E &\Rightarrow_{lm} E + E \\ &\Rightarrow_{lm} \text{id} + E \\ &\Rightarrow_{lm} \text{id} + (E) \\ &\Rightarrow_{lm} \text{id} + (E + E) \\ &\Rightarrow_{lm} \text{id} + (\text{id} + E) \end{aligned}$$

最左推导 (Left-most Derivation)

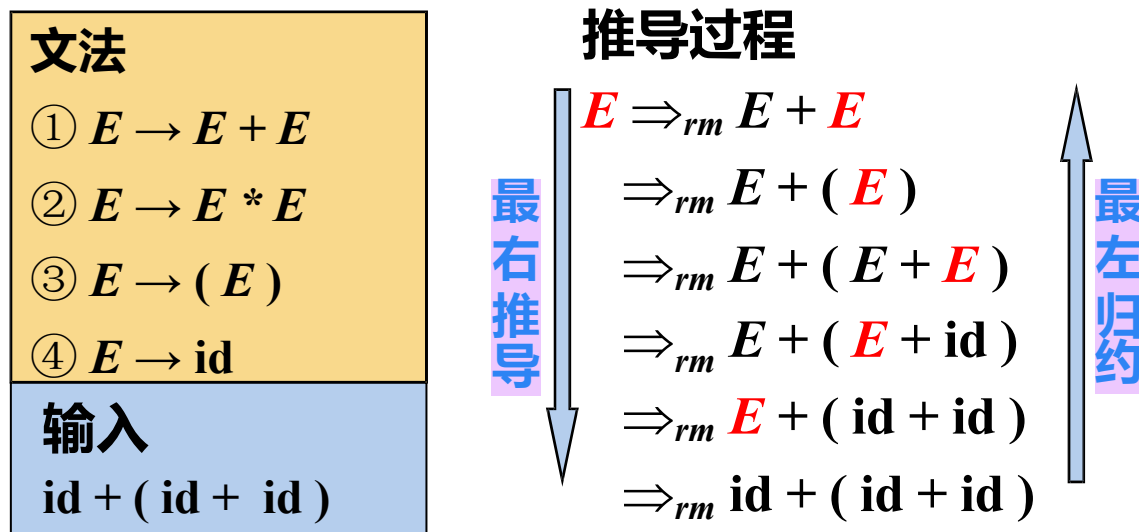
- **最左推导**: 每步代换**最左边的非终结符**



- 如果 $S \Rightarrow_{lm}^* \alpha_0$, 则称 α 是当前文法的**最左句型**(*left-sentential form*)
- 在自顶向下的分析中, 总是采用**最左推导**的方式

最右推导 (Right-most Derivation)

- **最右推导**: 每步代换**最右边的非终结符**



– 在自底向上的分析中，总是采用**最左归约**的方式

- **练习**: 尝试理解以上推导过程

句型、句子和语言

- **句型** (Sentential form): 对开始符号为 S 的文法 G , 如果 $S \Rightarrow^* \alpha$, $\alpha \in (T \cup N)^*$, 则称 α 是 G 的一个句型
 - 句型中既可以包含终结符, 又可以包含非终结符, 也可能是空串
- **句子** (Sentence): 如果 $S \Rightarrow^* w$, $w \in T^*$ 则称 w 是 G 的一个句子
 - 句子是不含非终结符的句型
 - 仅含终结符号的句型是一个句子
- **语言**: 由文法 G 推导出的所有句子构成的集合, 记为 $L(G)$ 。

$$L(G) = \{w \mid S \Rightarrow^* w, w \in T^*\}$$

语言 $L(G)$ 是由文法 G 产生的所有句子的集合

例: 文法定义的句型和句子

- **考虑文法:** $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

- **存在以下推导序列**

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$$

1. $-(\text{id} + \text{id})$ 是文法的句子(Sentence)
2. $-(\text{id} + E)$ 是文法的句型 (Sentential form)

思考题

- 上下文无关是什么意思？

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

- $A \rightarrow \gamma$ 是文法中的一个产生式

在文法推导的每一步，符号串 γ 仅依据 A 的产生式推导，无需依赖 A 的上下文 α 和 β ！

输入串的判定

问题：给定文法，如何判定输入串属于文法规定的语言？

从开始符号能推导出该词串

- 句子的**推导**（派生） - 从**生成**语言的角度
 - 句子的**归约** - 从**识别**语言的角度
- 从词串能归约出开始符号
- } 根据语法规则

文法：

- ① $\langle \text{句子} \rangle \rightarrow \langle \text{名词短语} \rangle \langle \text{动词短语} \rangle$
- ② $\langle \text{名词短语} \rangle \rightarrow \langle \text{形容词} \rangle \langle \text{名词短语} \rangle$
- ③ $\langle \text{名词短语} \rangle \rightarrow \langle \text{名词} \rangle$
- ④ $\langle \text{动词短语} \rangle \rightarrow \langle \text{动词} \rangle \langle \text{名词短语} \rangle$
- ⑤ $\langle \text{形容词} \rangle \rightarrow \textit{little}$
- ⑥ $\langle \text{名词} \rangle \rightarrow \textit{boy}$
- ⑦ $\langle \text{名词} \rangle \rightarrow \textit{apple}$
- ⑧ $\langle \text{动词} \rangle \rightarrow \textit{eat}$

推导

$\langle \text{句子} \rangle \Rightarrow \langle \text{名词短语} \rangle \langle \text{动词短语} \rangle$
 $\Rightarrow \langle \text{形容词} \rangle \langle \text{名词短语} \rangle \langle \text{动词短语} \rangle$
 $\Rightarrow \textit{little} \langle \text{名词短语} \rangle \langle \text{动词短语} \rangle$
 $\Rightarrow \textit{little} \langle \text{名词} \rangle \langle \text{动词短语} \rangle$
 $\Rightarrow \textit{little boy} \langle \text{动词短语} \rangle$
 $\Rightarrow \textit{little boy} \langle \text{动词} \rangle \langle \text{名词短语} \rangle$
 $\Rightarrow \textit{little boy eats} \langle \text{名词短语} \rangle$
 $\Rightarrow \textit{little boy eats} \langle \text{名词} \rangle$
 $\Rightarrow \textit{little boy eats apple}$

归约

2. 上下文无关文法

- CFG简介
- 推导和归约
- CFG的Parse Tree

分析树(Parse Tree): 推导的图形化表示

- 分析树具有下面性质

- 根节点为文法的**初始符号**
- 每个**叶子节点**是一个**终结符**
- 每个**内部节点**是一个**非终结符**
- 每个父节点及其子节点构成一条**产生式**

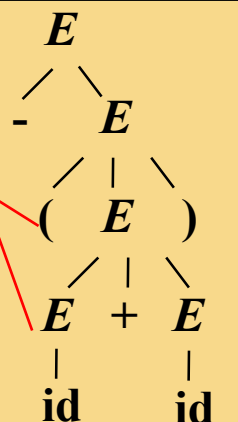
类似数理逻辑里面的
证明树(Proof tree!)

推导过程： $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

文法:

- ① $E \rightarrow E + E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow -E$
- ④ $E \rightarrow (E)$
- ⑤ $E \rightarrow id$

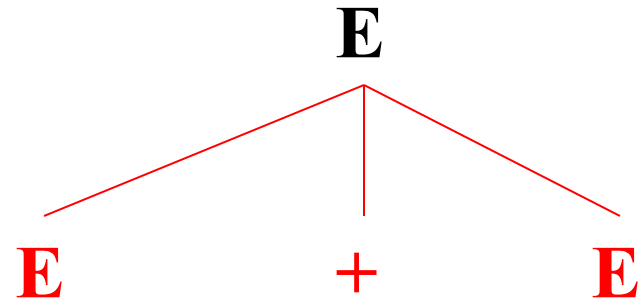
分析树:



例: Parse Tree构造

- 文法 $E \rightarrow E + E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$

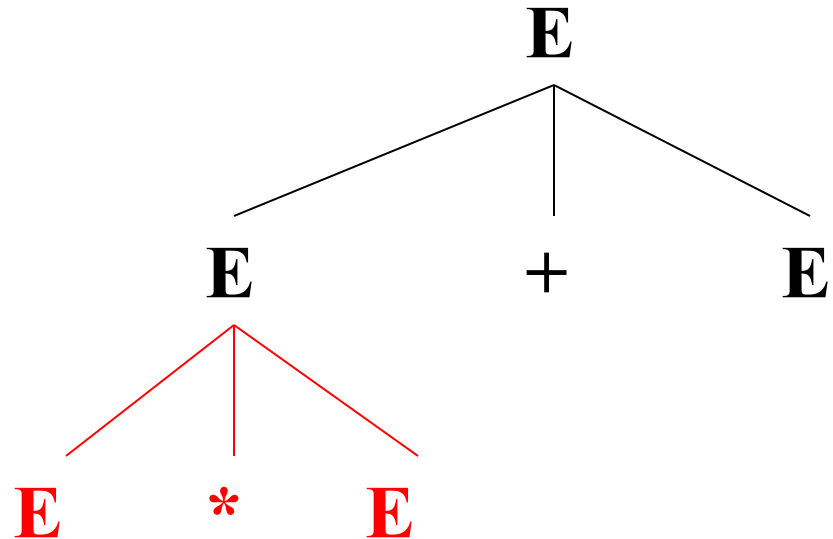
E
 $\rightarrow E + E$



例: Parse Tree构造

- 文法 $E \rightarrow E+E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$

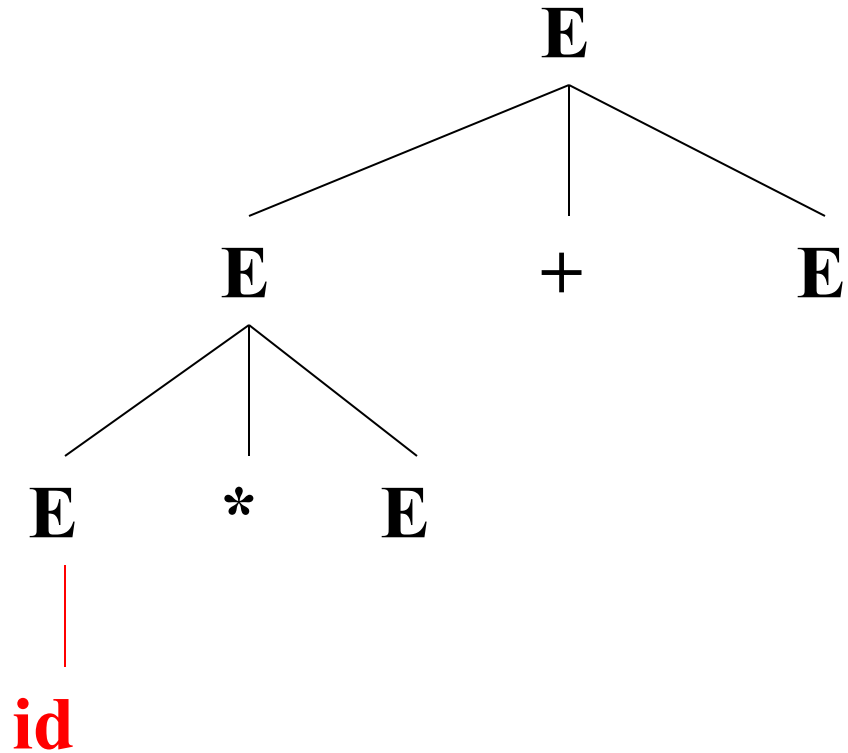
E
 $\rightarrow E+E$
 $\rightarrow E * E+E$



例: Parse Tree构造

- 文法 $E \rightarrow E + E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$

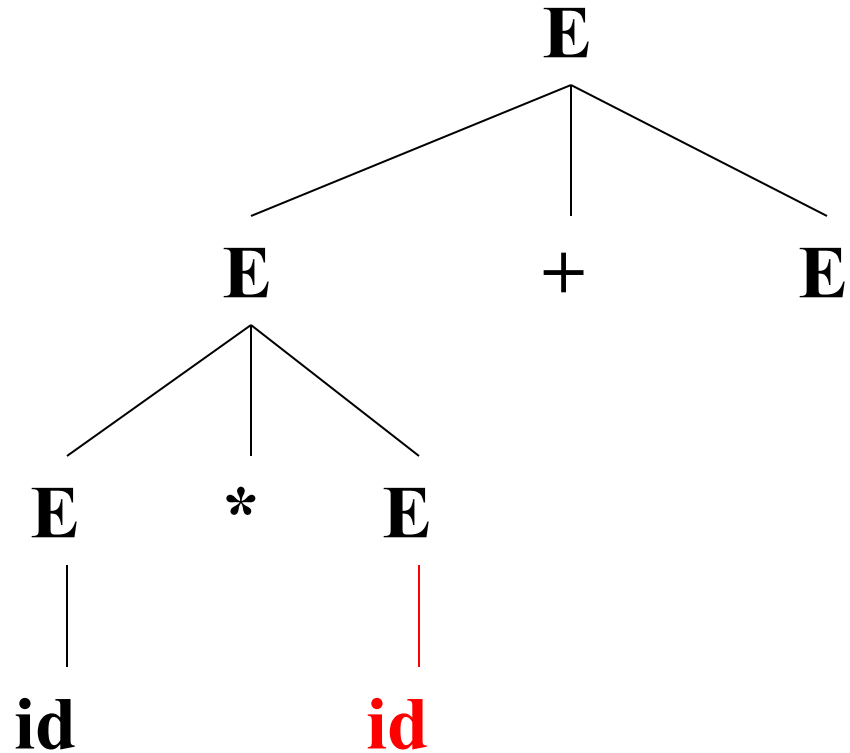
E
 $\rightarrow E + E$
 $\rightarrow E * E + E$
 $\rightarrow id * E + E$



例: Parse Tree构造

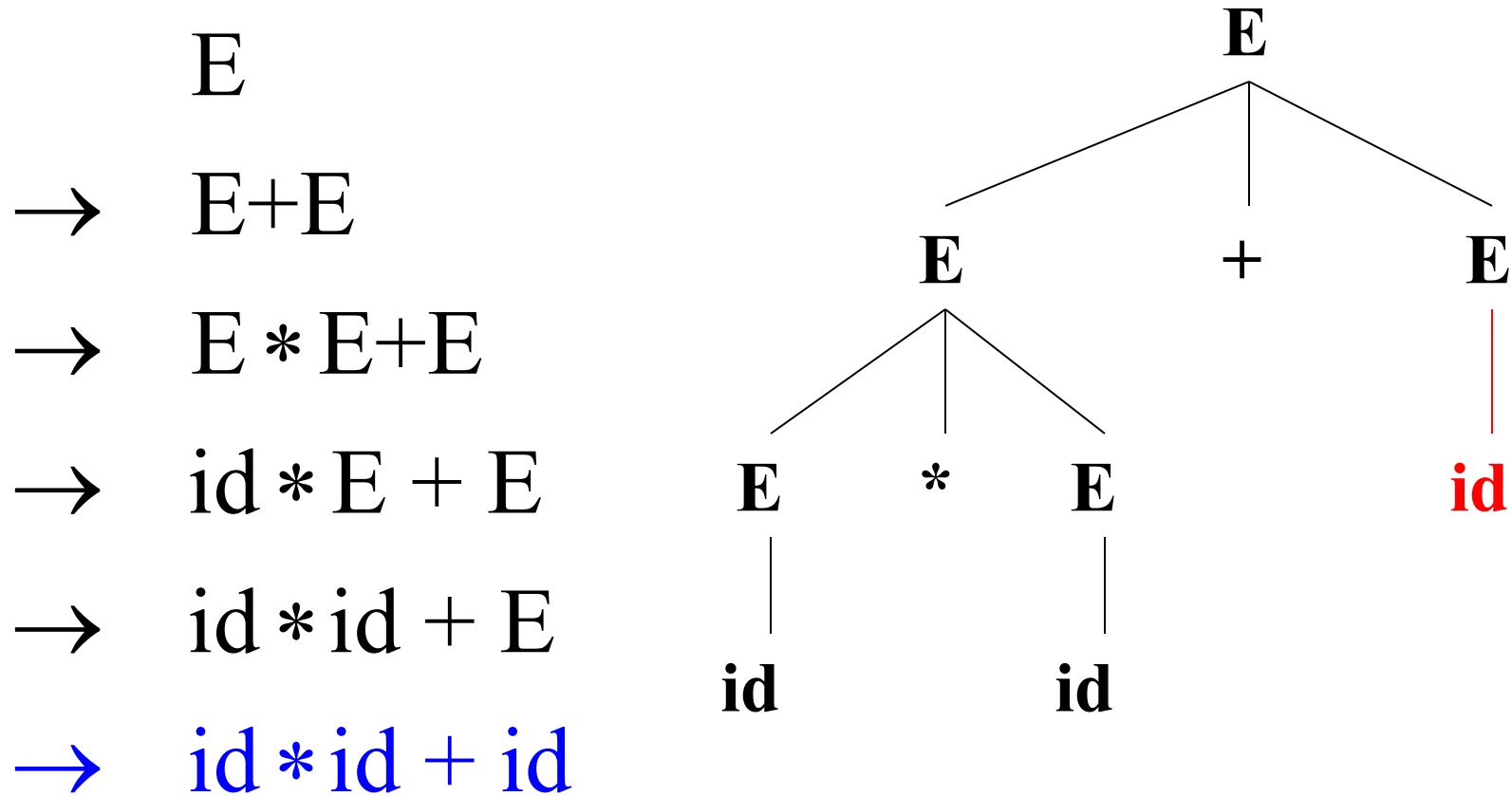
- 文法 $E \rightarrow E+E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$

E
 $\rightarrow E+E$
 $\rightarrow E * E+E$
 $\rightarrow id * E + E$
 $\rightarrow id * id + E$



例: Parse Tree构造

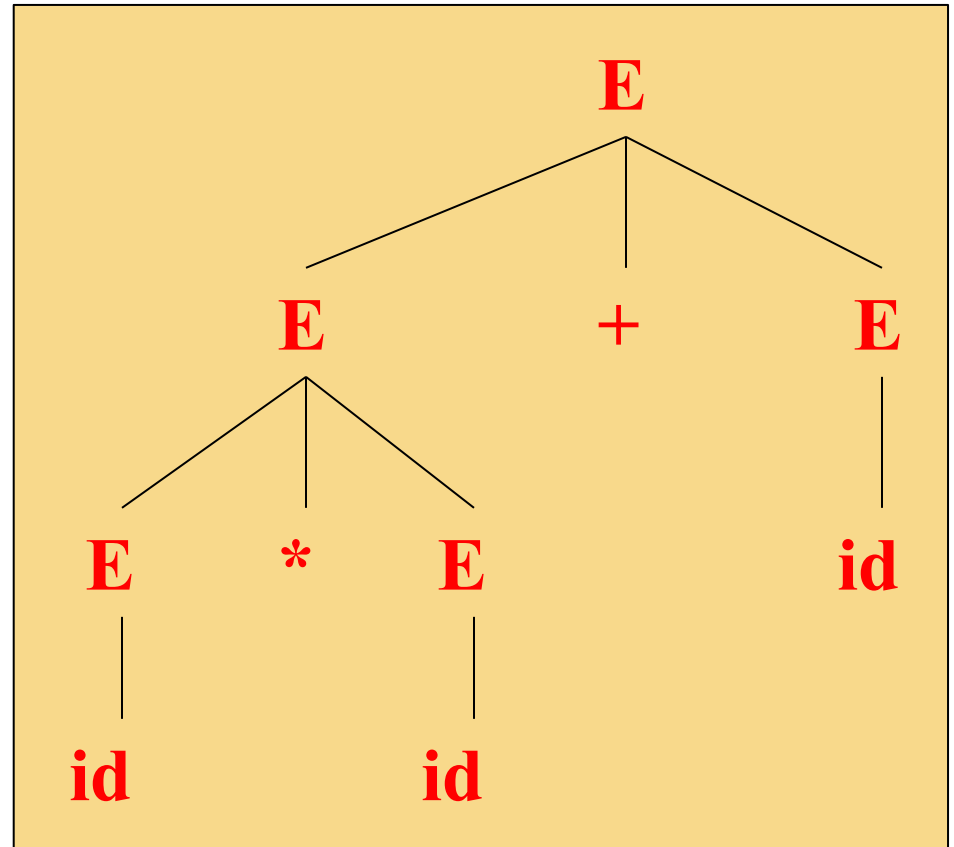
- 文法 $E \rightarrow E+E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$



例: Parse Tree构造

- 文法 $E \rightarrow E+E \mid E * E \mid (E) \mid id$
- 串 $id * id + id$

E
 $\rightarrow E+E$
 $\rightarrow E * E+E$
 $\rightarrow id * E + E$
 $\rightarrow id * id + E$
 $\rightarrow id * id + id$



练习: Parse Tree构造

- 对于文法 $expr \rightarrow term \mid term + expr \mid term - expr$
 $term \rightarrow num \mid (expr)$
- 展示 $1 + (2 - 3)$ 的构造过程

expr

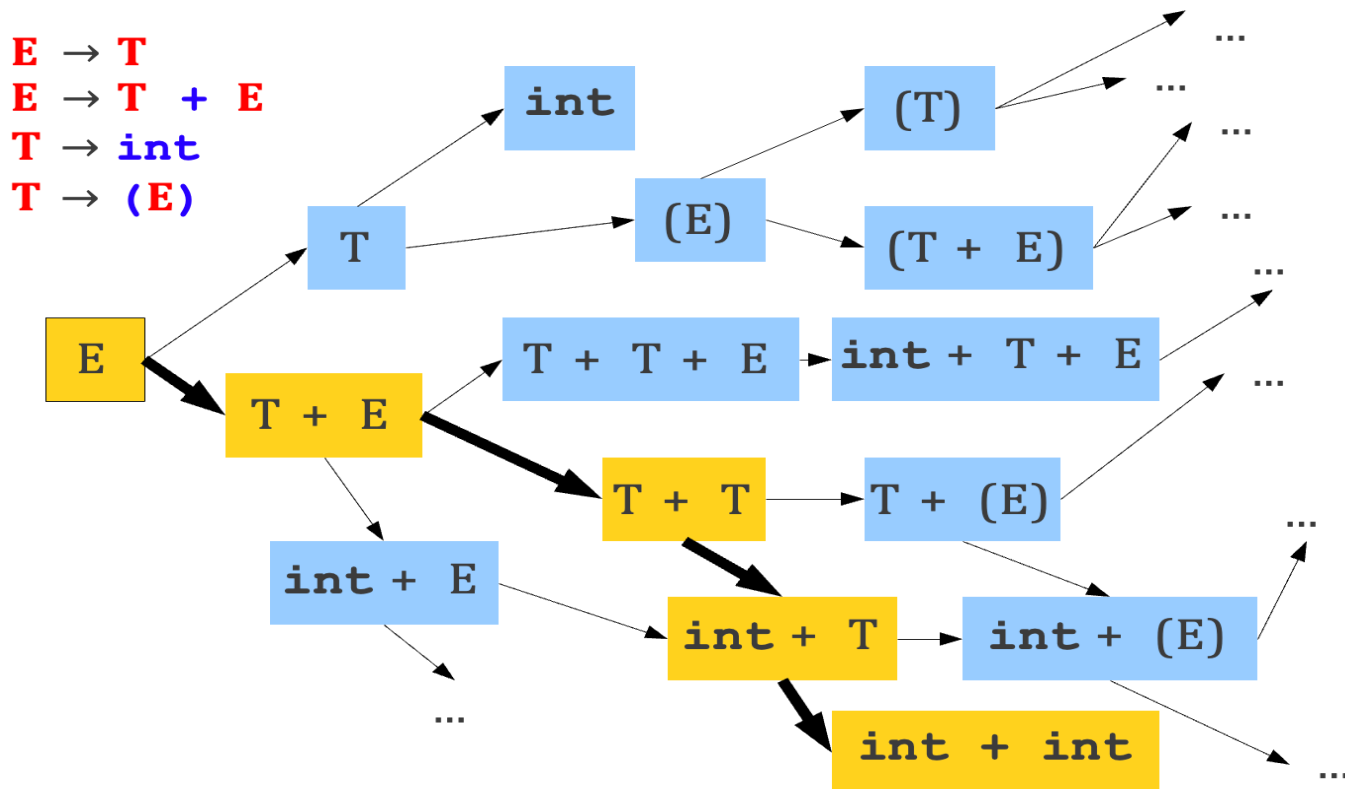
expr

语法分析作为搜索问题

对于一个终结符号串 x , 从 S 推导出 x 或者将 x 归约为 S

- 句子的推导
- 句子的归约

- 从生成语言的角度
- 从识别语言的角度 } 根据**文法规则**



语法分析的复杂度

对于一个终结符号串 x , 从 S 推导出 x 或者将 x 归约为 S

- If there are no restrictions on the form of grammar used, parsing CFL requires $O(n^3)$ time
 - E.g., Cocke-Younger-Kasami's algorithm (CYK)
- Subsets of CFLs typically require $O(n)$ time
 - Predictive parsing using LL(1) grammars
 - Shift-Reduce parsing using LR(1) grammars

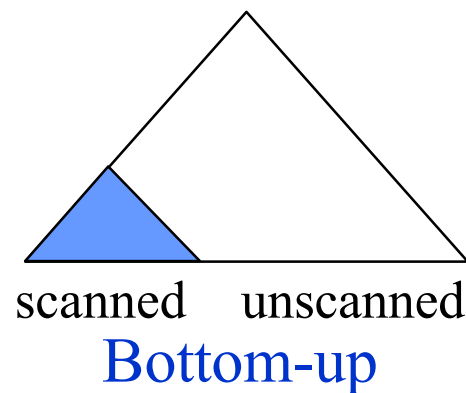
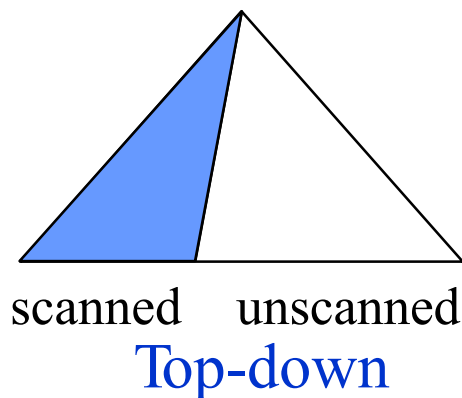


为了实现高效搜索, 控制搜索空间, 如文法产生式的限制

语法分析算法分类

对于一个终结符号串 x , 从 S 推导出 x 或者将 x 归约为 S

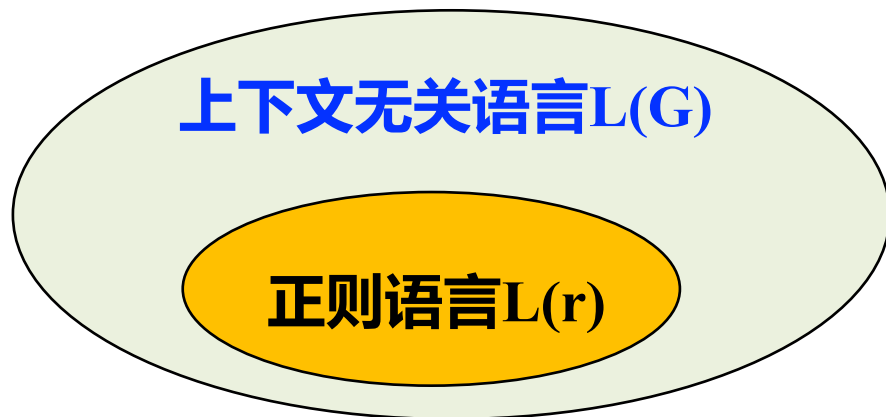
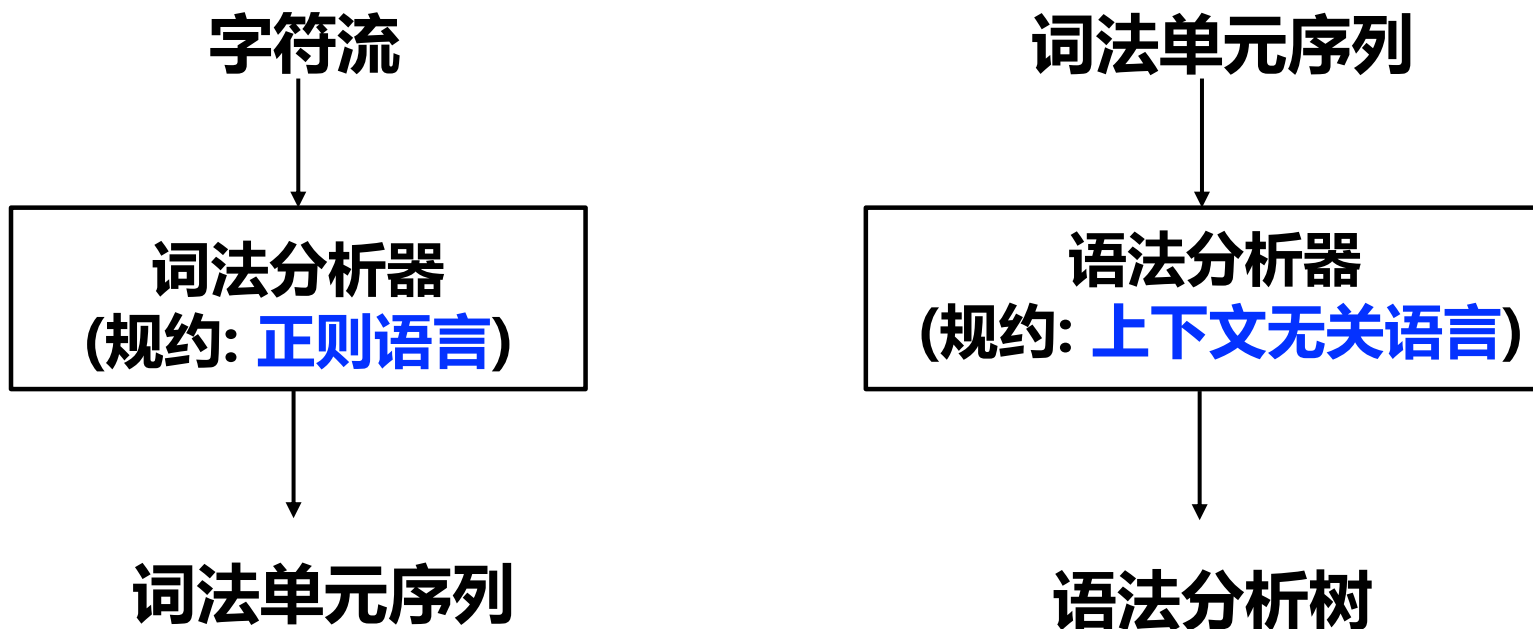
- **自顶向下(Top-down)分析 (LL系列)**
 - 从开始符号 S 出发, 根据产生式规则推导出 x
 - Parse Tree的构造方法: 从根部开始
- **自底向上(Bottom-up)分析 (LR系列)**
 - 根据产生式规则归约(reduce)到文法的开始符号 S
 - Parse Tree的构造方法: 从叶子开始



3. 设计编程语言的文法

- 对比RE和CFG
- CFG的改造/限制

语法分析 vs 词法分析



回顾: 形式语言

- **语言**：字母表 Σ 上的一个串集

- 例: $\{\varepsilon, 0, 00, 000, \dots\}$, $\{\varepsilon\}$, $\emptyset$

- **句子**：属于语言的串

- **语言的运算**

运算	定义和表示
L 和 M 的并	$L \cup M = \{s \mid s \text{ 属于 } L \text{ 或者 } s \text{ 属于 } M\}$
L 和 M 的连接	$LM = \{st \mid s \text{ 属于 } L \text{ 且 } t \text{ 属于 } M\}$
L 的幂	$\begin{cases} L^0 = \{\varepsilon\} \\ L^n = L^{n-1}L, n \geq 1 \end{cases}$
L 的Kleene闭包	$L^* = \bigcup_{i=0}^{\infty} L^i$
L 的正闭包	$L^+ = \bigcup_{i=1}^{\infty} L^i$

回顾: 正则语言--由正则表达式刻画

- 正则表达式 r 定义正则语言, 记为 $L(r)$

- ε 是一个 RE , $L(\varepsilon) = \{\varepsilon\}$
- 如果 $a \in \Sigma$, 则 a 是一个 RE , $L(a) = \{a\}$
- 假设 r 和 s 都是 RE , 分别表示语言 $L(r)$ 和 $L(s)$
 - $r|s$ 是一个 RE , $L(r|s) = L(r) \cup L(s)$
 - rs 是一个 RE , $L(rs) = L(r) L(s)$
 - r^* 是一个 RE , $L(r^*) = (L(r))^*$ Kleene 闭包
 - (r) 是一个 RE , $L((r)) = L(r)$

回顾: 上下文无关语言

- **上下文无关文法(CFG)的定义**

$$G = (T, N, P, S)$$

T : 终结符集合

N : 非终结符集合

P : 产生式(Production) $A \rightarrow \alpha$ 集合

– **约束**: 左部 A 是一个**非终结符号**, 右部 α 是符号串

S : 开始符号 (Start symbol): 某个非终结符号

上下文无关语言 $L(G)$: 由文法 G 产生的所有句子的集合

正则语言的形式文法刻画!

- **正则文法的定义**

$$G = (T, N, P, S)$$

T : 终结符集合

N : 非终结符集合

P : 产生式(Production) $\alpha \rightarrow \beta$ 集合

– **右线性文法**: $\alpha \rightarrow \beta$ 形如 $A \rightarrow aB$ 或 $A \rightarrow a$ 其中 $A, B \in N$,
 $a \in T \cup \{\varepsilon\}$

– **左线性文法**: $\alpha \rightarrow \beta$ 形如 $A \rightarrow Ba$ 或 $A \rightarrow a$

S : 开始符号 (start symbol): 某个非终结符号

正则语言: 右线性文法/左线性文法产生的所有句子的集合

思考: 为什么词法分析用正则表达式, 不用正则文法?

- 正则表达式描述简洁(刻画Token), 且易于理解

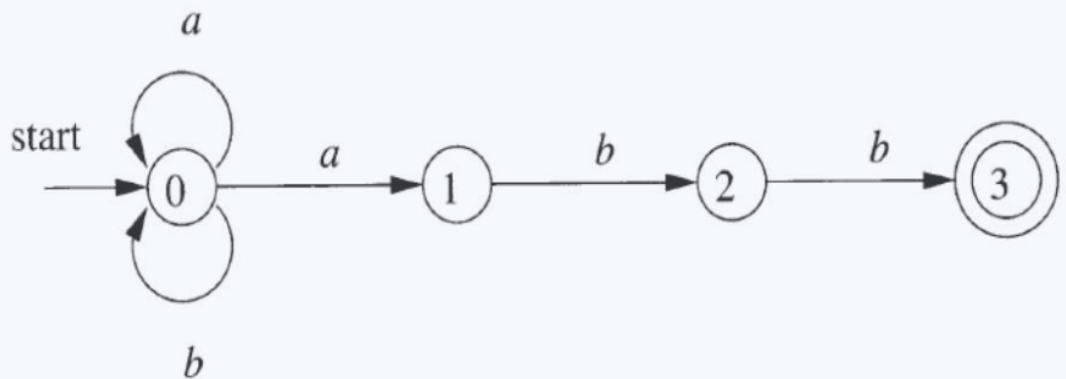
$A_0 \rightarrow aA_0 \mid aA_1 \mid bA_0$

$A_1 \rightarrow bA_2$

$A_2 \rightarrow bA_3$

$A_3 \rightarrow \varepsilon$

正则文法定义



NFA状态转换图

$(a|b)^* abb$

正则表达式定义

正则语言 vs 上下文无关语言

- 从文法角度，对产生式 $\alpha \rightarrow \beta$ 形式的限制不同
 - 上下文无关文法：
左部 α 是一个非终结符号，右部是一个**符号串**
 - 正则文法：
右线性 $A \rightarrow aB$ 或 $A \rightarrow a$, 左线性 $A \rightarrow Ba$ 或 $A \rightarrow a$



每个正则语言都是一个上下文无关语言，反之不成立

正则语言 vs 上下文无关语言

- 正则语言表达能力有限，难以刻画编程语言的语法
 - 如：不能用于描述配对或嵌套的结构
 - 例：配对括号串的集合，如不能表达 $(^n)^n, n \geq 1$
 - 原因：有穷自动机无法记录访问同一状态的次数

```
int foo(int x){  
    int y;  
    if ((x > 0)  
        y = 1;  
    else  
        y = 0  
    return y;  
}
```

```
foo.c:4:9: error: expected ')'  
        y = 1;  
        ^  
foo.c:3:8: note: to match this '('  
    if ((x > 0)  
        ^  
foo.c:6:14: error: expected ';' after expression  
        y = 0  
           ^  
           ;  
2 errors generated._
```

引申: 形式文法的分类 (不要求掌握)

Chomsky在1956创立了形式语言学，并将形式语言的文法分为四类：

文法 $G = (T, N, P, S)$

- **0型文法(短语结构文法): 递归可枚举语言**
 - $\alpha \rightarrow \beta$, $\alpha, \beta \in (N \cup T)^*$, $|\alpha| \geq 1$
- **1型文法(上下文有关文法)**
 - $|\alpha| \leq |\beta|$, 但 $S \rightarrow \varepsilon$ 可以例外
- **2型文法(上下文无关文法)**
 - $A \rightarrow \beta$, $A \in N$, $\beta \in (N \cup T)^*$
- **3型文法(正则文法)**
 - $A \rightarrow aB$ (左线性 $A \rightarrow Ba$) , 或 $A \rightarrow a$, $A, B \in N$, $a \in T$

3. 设计编程语言的文法

- 对比RE和CFG
- CFG的改造/限制

设计编程语言的文法

- 为了高效语法分析，可能对文法做处理/限制

- 消除**二义性** (Resolving ambiguities)

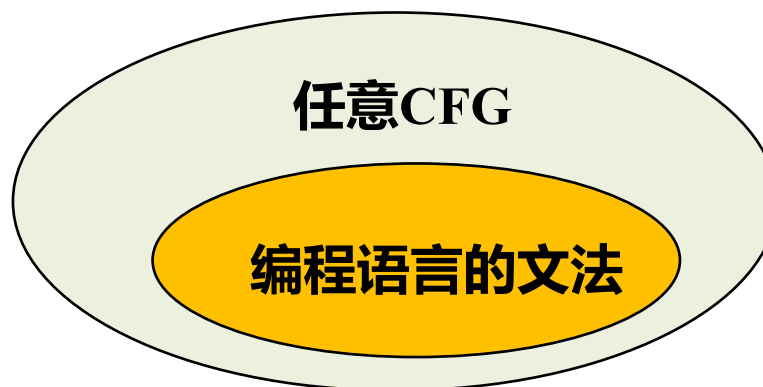
- 消除左递归(Elimination of left recursions)

- Avoid infinite loop in top-down parsing

- 提左公因子(Left-factoring)

- Avoid backtracking in top-down parsing

} 通常限于自顶向下分析
(后续会讲)



二义性文法 (Ambiguous Grammar)

- **二义性**: 如果文法的某些句子存在**不止一棵**分析树, 则该文法是二义的
 - 或者说, 若一个文法中存在某个句子, 它有两个不同的最左(最右)推导, 则这个文法是二义性的
- 二义性会给语法分析带来不确定性, 程序设计语言的文法通常都应该是无二义性的

例: 二义性文法

• 文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

• $\text{id} * \text{id} + \text{id}$ 有两个不同的最左推导

$$E \Rightarrow E * E$$

$$\Rightarrow \text{id} * E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

$$E \Rightarrow E + E$$

$$\Rightarrow E * E + E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

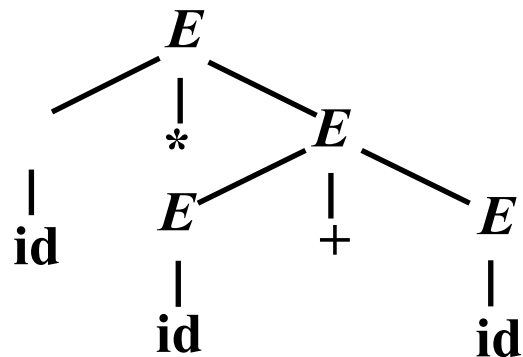
$$\Rightarrow \text{id} * \text{id} + \text{id}$$

例: 二义性文法

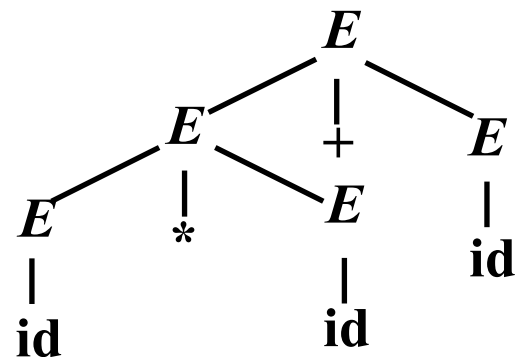
• 文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

• $\text{id} * \text{id} + \text{id}$ 有两棵不同的分析树

$E \Rightarrow E * E$
 $\Rightarrow \text{id} * E$
 $\Rightarrow \text{id} * E + E$
 $\Rightarrow \text{id} * \text{id} + E$
 $\Rightarrow \text{id} * \text{id} + \text{id}$



$E \Rightarrow E + E$
 $\Rightarrow E * E + E$
 $\Rightarrow \text{id} * E + E$
 $\Rightarrow \text{id} * \text{id} + E$
 $\Rightarrow \text{id} * \text{id} + \text{id}$



二义性的影响

- 编程语言的文法通常是**无二义**的
- 否则就会导致一个程序有多种“正确”的解释

$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$$

$$E \Rightarrow E * E$$

$$\Rightarrow \text{id} * E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

Wrong!

$$E \Rightarrow E + E$$

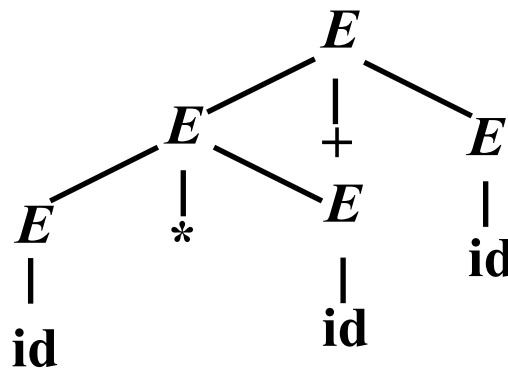
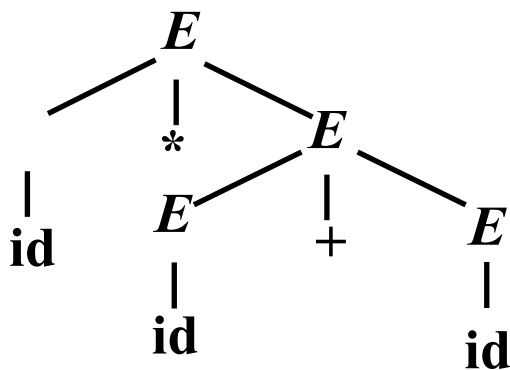
$$\Rightarrow E * E + E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

Right



消除二义性?

- **二义性**: 如果文法的某些句子存在**不止一棵**分析树, 则该文法是二义的
- **难点**: “给定CFG是否无二义性”是不可判定问题! [1]
- 关于二义性处理的近期相关研究工作 [2]

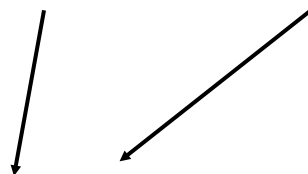
[1] On The Ambiguity Problem of Backus Systems. JACM'62

[2] Statically Resolvable Ambiguity, POPL'23

消除二义性

- 文法产生二义性的根源
 - 多种 “正确” 推导处于文法同一层

$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$$



+, *操作都是左结合的，并且在运算中有不同的优先级，但是在这个文法中没有得到体现

消除二义性

- 文法产生二义性的根源
 - 多种“正确”推导处于文法同一层
- 消除二义性的惯用技术：分层
 - 规定符号的优先级
 - 规定符号的结合性

$$\begin{aligned} E &\rightarrow E + E \\ &\quad | E * E \\ &\quad | (E) | \text{id} \end{aligned}$$


+ , * 操作都是左结合的，并且*比+有更高的优先级

消除二义性

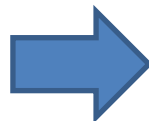
- **运算优先级**

- 根据算符不同的优先级，引入新的非终结符
- 越接近开始符号 S 的文法符号优先级越低

- **运算结合性**

- 递归非终结符在终结符左边，运算就左结合
- 如 $A \rightarrow A\beta$ ， A 在终结符(如 $+$, $*$)左侧出现(即终结符在 β 中)

$$\begin{aligned} E &\rightarrow E + E \\ &\quad | E * E \\ &\quad | (E) | \text{id} \end{aligned}$$



$$\begin{aligned} E &\rightarrow E + T | T \\ T &\rightarrow T * F | F \\ F &\rightarrow (E) | \text{id} \end{aligned}$$

$+$ ， $*$ 操作都是**左结合**的，并且 $*$ 比 $+$ 有更高的**优先级**

练习: 消除二义性

- **悬空else文法**

$$\begin{aligned} E \rightarrow & \text{if } E \text{ then } E \\ & | \text{if } E \text{ then } E \text{ else } E \\ & | \text{OTHER} \end{aligned}$$

- 判断该文法有无二义性
- 如果存在二义性，如何消除

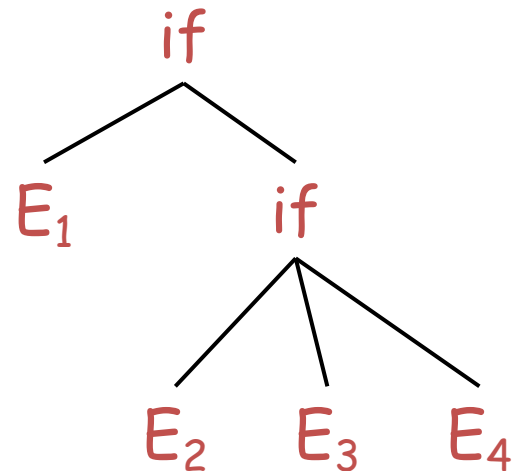
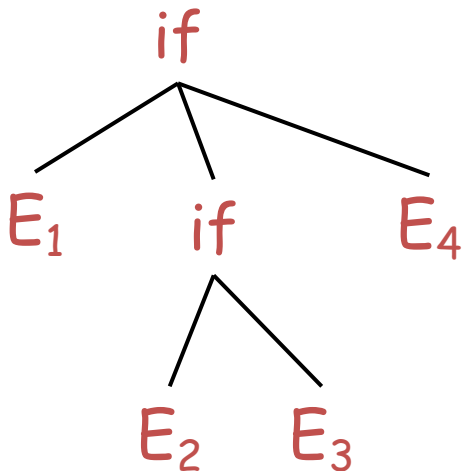
考虑句型: $\text{if } E_1 \text{ then if } E_2 \text{ then } E_3 \text{ else } E_4$

练习: 消除二义性

- 悬空else文法

$E \rightarrow \text{if } E \text{ then } E$
 $\quad \quad \quad | \text{if } E \text{ then } E \text{ else } E$
 $\quad \quad \quad | \text{OTHER}$

- $\text{if } E_1 \text{ then if } E_2 \text{ then } E_3 \text{ else } E_4$ 的2个parse tree:



练习: 消除二义性

- **else** matches the closest unmatched **then**
- We can describe this in the grammar (distinguish between matched and unmatched “then”)

```
E → MIF          /* all then are matched */  
   | UIF          /* some then are unmatched */  
  
MIF → if E then MIF else MIF  
     | OTHER  
  
UIF → if E then E  
     | if E then MIF else UIF
```

- Describes the same set of strings

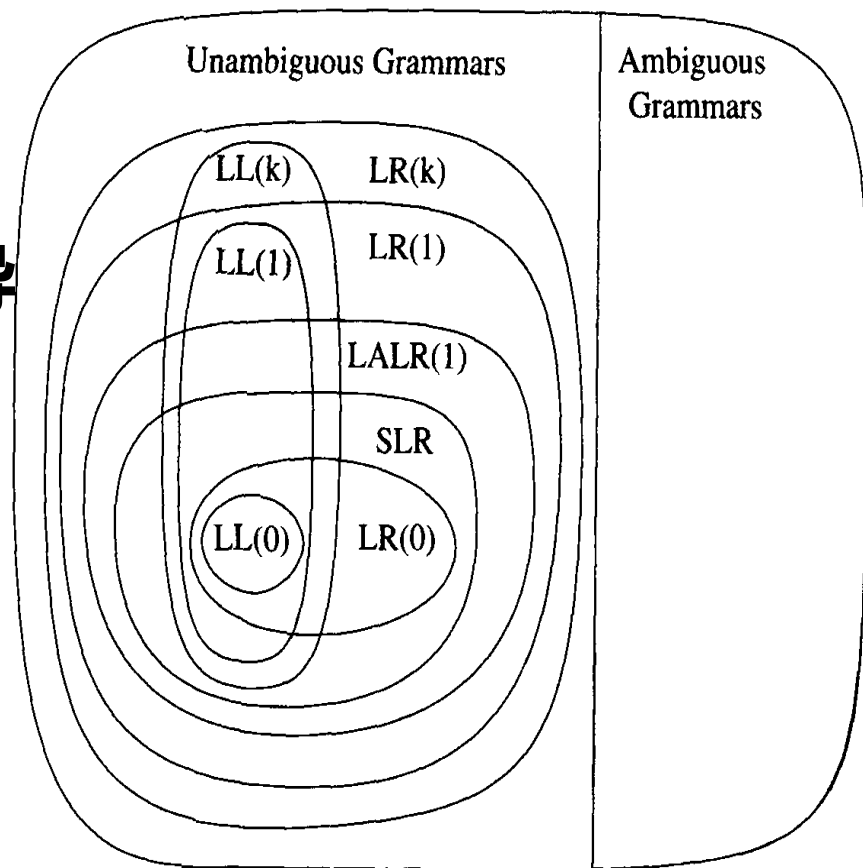
Precedence and Associativity Declarations

- Instead of rewriting the grammar
 - Use the more natural (ambiguous) grammar
 - Along with disambiguating declarations
- Most tools allow **precedence and associativity declarations** to disambiguate grammars
 - 在 Yacc (Parser自动生成器)等工具中，我们可以直接指定优先级、结合性而无需自己重写文法

```
1    %left '+'
2    %left '*'
3    %right '-'
```

本讲小结

- CFG及其定义的语言
- 推导和归约、最左/右推导
- CFG的Parse Tree
- 设计编程语言的文法





Thank you all for your attention

一些记号

通常的符号约定(供参考，不用记忆)

- **终结符 (Terminals, T)**
 - (a) 字母表中**排在前面的小写字母**，如 a 、 b 、 c
 - (b) **运算符**，如 $+$ 、 $*$ 等
 - (c) **标点符号**，如括号、逗号等
 - (d) **数字**0、1、...、9
 - (e) **粗体字符串**，如id、if等

通常的符号约定

- **终结符(Terminals, T)**
- **非终结符 (Nonterminals , N)**
 - (a) 字母表中**排在前面的大写字母** , 如 A 、 B 、 C
 - (b) 字母 S 。通常表示开始符号
 - (c) **小写、斜体的名字** , 如 $expr$ 、 $stmt$ 等
 - (d) **代表程序构造的大写字母**。如 E (表达式)、 T (项)

通常的符号约定

- **终结符**

- **非终结符**

终结符	a, b, c	终结符号串	$u, v, \dots, z$
非终结符	A, B, C		
文法符号	X, Y, Z	文法符号串	α, β, γ

- 字母表中**排在后面的大写字母**（如 X 、 Y 、 Z ）表示**文法符号**（即终结符或非终结符）
- 字母表中**排在后面的小写字母**（主要是 u 、 v 、 $\dots$ 、 z ）表示**终结符号串**（包括**空串**）
- **小写希腊字母**，如 α 、 β 、 γ ，表示**文法符号串**（包括**空串**）
- 除非特别说明，**第一个产生式的左部**就是**开始符号**